Orchestrate teams of Claude Code sessions

Coordinate multiple Claude Code instances working together as a team, with shared tasks, inter-agent messaging, and centralized management.

> ***Warning:*** *Agent teams are experimental and disabled by default. Enable them by adding* `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS` *to your settings.json or environment. Agent teams have known limitations around session resumption, task coordination, and shutdown behavior.*

Agent teams let you coordinate multiple Claude Code instances working together. One session acts as the team lead, coordinating work, assigning tasks, and synthesizing results. Teammates work independently, each in its own context window, and communicate directly with each other.

Unlike subagents, which run within a single session and can only report back to the main agent, you can also interact with individual teammates directly without going through the lead.

> ***Note:*** *Agent teams require Claude Code v2.1.32 or later. Check your version with* `claude --version`.

This page covers: when to use agent teams, starting a team, controlling teammates, and best practices for parallel work.

When to use agent teams

Agent teams are most effective for tasks where parallel exploration adds real value. The strongest use cases are:

- **Research and review:** multiple teammates can investigate different aspects of a problem simultaneously
- **New modules or features:** teammates can each own a separate piece without stepping on each other
- **Debugging with competing hypotheses:** teammates test different theories in parallel
- **Cross-layer coordination:** changes spanning frontend, backend, and tests

Agent teams add coordination overhead and use significantly more tokens than a single session. They work best when teammates can operate independently. For sequential tasks, same-file edits, or work with many dependencies, a single session or subagents are more effective.

Compare with subagents

Both agent teams and subagents let you parallelize work, but they operate differently. Choose based on whether your workers need to communicate with each other.

	Subagents	Agent teams
Context	Own context; results return to caller	Own context; fully independent
Communication	Report results to main agent only	Teammates message each other directly
Coordination	Main agent manages all work	Shared task list; self-coordinating
Best for	Focused tasks, only result matters	Complex collaborative work
Token cost	Lower — results summarized back	Higher — separate Claude instance each

Enable agent teams

Enable agent teams by setting `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1` in your environment or in settings.json:

```
{
```

```
  "env": {
    "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1"
  }
}
```

Start your first agent team

After enabling agent teams, tell Claude to create a team and describe the task structure you want in natural language. Claude creates the team, spawns teammates, and coordinates work. Example:

```
I'm designing a CLI tool that helps developers track TODO comments across
their codebase. Create an agent team to explore this from different angles:
one teammate on UX, one on technical architecture, one playing devil's advocate.
```

The lead's terminal lists all teammates and what they're working on. Use **Shift+Down** to cycle through teammates and message them directly.

Control your agent team

Tell the lead what you want in natural language. It handles team coordination, task assignment, and delegation based on your instructions.

Choose a display mode

- **In-process:** all teammates run inside your main terminal. Use Shift+Down to cycle and type to message them. Works in any terminal.
- **Split panes:** each teammate gets its own pane. Requires tmux or iTerm2.

The default is "auto" — split panes if inside tmux, otherwise in-process. Override in settings.json:

```
{ "teammateMode": "in-process" }
```

Or pass as a flag: `claude --teammate-mode in-process`

Specify teammates and models

```
Create a team with 4 teammates to refactor these modules in parallel.
Use Sonnet for each teammate.
```

Require plan approval for teammates

For risky tasks, require teammates to plan before implementing. The teammate works in read-only plan mode until the lead approves their approach.

```
Spawn an architect teammate to refactor the authentication module.
Require plan approval before they make any changes.
```

Talk to teammates directly

- **In-process mode:** Shift+Down to cycle, then type to send. Enter to view a session, Escape to interrupt. Ctrl+T to toggle the task list.
- **Split-pane mode:** click into a teammate's pane to interact directly.

Assign and claim tasks

The shared task list coordinates work across the team. Tasks have three states: pending, in progress, and completed. Tasks can depend on other tasks. Task claiming uses file locking to prevent race conditions.

Shut down teammates

```
Ask the researcher teammate to shut down
```

Clean up the team

```
Clean up the team
```

> ***Important:*** *Always use the lead to clean up. Teammates should not run cleanup because their team context may not resolve correctly.*

Enforce quality gates with hooks

- **TeammateIdle:** runs when a teammate is about to go idle. Exit with code 2 to send feedback and keep working.
- **TaskCompleted:** runs when a task is being marked complete. Exit with code 2 to prevent completion.

How agent teams work

Architecture

Component	Role
Team lead	Main Claude Code session; creates team, spawns teammates, coordinates work
Teammates	Separate Claude Code instances; each works on assigned tasks
Task list	Shared list of work items that teammates claim and complete
Mailbox	Messaging system for communication between agents

Teams and tasks are stored locally at `~/.claude/teams/{team-name}/` and `~/.claude/tasks/{team-name}/`.

Permissions

Teammates start with the lead's permission settings. If the lead runs with `--dangerously-skip-permissions`, all teammates do too. You can change individual teammate modes after spawning.

Context and communication

Each teammate has its own context window. When spawned, a teammate loads the same project context (CLAUDE.md, MCP servers, skills) but does not inherit the lead's conversation history.

- **Automatic message delivery:** messages are delivered automatically to recipients.
- **Idle notifications:** teammates automatically notify the lead when they finish.
- **Shared task list:** all agents can see task status and claim available work.
- **message:** send to one specific teammate. **broadcast:** send to all (use sparingly).

Token usage

Agent teams use significantly more tokens than a single session. Token usage scales with the number of active teammates. For research and new feature work, the extra tokens are usually worthwhile; for routine tasks, a single session is more cost-effective.

Use case examples

Run a parallel code review

```
Create an agent team to review PR #142. Spawn three reviewers:
- One focused on security implications
- One checking performance impact
- One validating test coverage
```

```
Have them each review and report findings.
```

Each reviewer works from the same PR but applies a different filter. The lead synthesizes findings across all three after they finish.

Investigate with competing hypotheses

```
Users report the app exits after one message instead of staying connected.
Spawn 5 agent teammates to investigate different hypotheses. Have them talk
to each other to try to disprove each other's theories, like a scientific
debate. Update the findings doc with whatever consensus emerges.
```

The debate structure forces multiple independent investigators to actively challenge each other. The theory that survives is much more likely to be the actual root cause.

Best practices

Give teammates enough context

Include task-specific details in the spawn prompt since teammates don't inherit the lead's conversation history.

```
Spawn a security reviewer: 'Review src/auth/ for security vulnerabilities.
Focus on token handling, session management, and input validation.
The app uses JWT tokens in httpOnly cookies. Report with severity ratings.'
```

Choose an appropriate team size

- Token costs scale linearly — each teammate consumes tokens independently.
- Coordination overhead increases with more teammates.
- Start with 3–5 teammates for most workflows.
- Having 5–6 tasks per teammate keeps everyone productive without excessive context switching.

Size tasks appropriately

- **Too small:** coordination overhead exceeds the benefit
- **Too large:** teammates work too long without check-ins, increasing wasted effort
- **Just right:** self-contained units with a clear deliverable (a function, test file, or review)

Avoid file conflicts

Two teammates editing the same file leads to overwrites. Break the work so each teammate owns a different set of files.

Monitor and steer

Check in on teammates' progress, redirect approaches that aren't working, and synthesize findings as they come in. Letting a team run unattended for too long increases the risk of wasted effort.

Troubleshooting

Teammates not appearing

- In in-process mode, press Shift+Down to cycle through active teammates.
- Check the task was complex enough to warrant a team.
- For split panes, ensure tmux is installed: `which tmux`
- For iTerm2, verify the it2 CLI is installed and the Python API is enabled.

Too many permission prompts
Pre-approve common operations in your permission settings before spawning teammates.

Teammates stopping on errors
Check their output using Shift+Down or by clicking the pane, then give additional instructions or spawn a replacement teammate.

Orphaned tmux sessions
```
tmux ls
tmux kill-session -t <session-name>
```

Limitations
Agent teams are experimental. Current known limitations:

- **No session resumption with in-process teammates:** /resume and /rewind do not restore in-process teammates.
- **Task status can lag:** teammates sometimes fail to mark tasks as completed, which blocks dependent tasks.
- **Shutdown can be slow:** teammates finish their current request before shutting down.
- **One team per session:** clean up the current team before starting a new one.
- **No nested teams:** teammates cannot spawn their own teams. Only the lead can manage the team.
- **Lead is fixed:** you can't promote a teammate to lead or transfer leadership.
- **Permissions set at spawn:** all teammates start with the lead's permission mode.
- **Split panes require tmux or iTerm2:** not supported in VS Code terminal, Windows Terminal, or Ghostty.

Next steps
- **Lightweight delegation:** subagents spawn helper agents for research or verification within your session.
- **Manual parallel sessions:** Git worktrees let you run multiple Claude Code sessions yourself.
- **Compare approaches:** see the subagent vs agent team comparison for a side-by-side breakdown.
